

Apache Spark Overview

1. What Is Apache Spark?

Apache Spark is a distributed data processing engine designed to process large datasets across a cluster. A Spark application can distribute computation over multiple executors while a driver coordinates execution.

Spark is commonly used for batch ETL, SQL analytics, streaming, machine learning pipelines, and large-scale data transformation. Its APIs include PySpark, Scala, Java, and SQL.

2. Core Architecture

A Spark application consists of a driver and one or more executors. The driver creates the execution plan, coordinates work, and tracks application state. Executors run tasks and can cache data in memory or on disk.

Spark applications are divided into jobs, stages, and tasks. An action such as count, collect, or write usually triggers a job. Stages represent groups of tasks separated by shuffle boundaries.

3. Transformations and Actions

Transformations create a new logical dataset without immediately executing the computation. Examples include select, filter, map, join, groupBy, and withColumn.

Actions request a result or persist data. Examples include count, collect, show, write, and saveAsTable. Spark uses lazy evaluation so it can optimize a chain of transformations before execution.

4. RDDs, DataFrames, and Datasets

RDDs are Spark's low-level distributed collection abstraction. DataFrames provide named columns and allow Spark SQL's optimizer to reason about relational operations. Datasets are available primarily in Scala and Java and combine typed APIs with Spark SQL optimization.

For most modern ETL work, DataFrames and Spark SQL are preferred because they enable optimizer-based execution and concise transformations.

5. Spark SQL and Catalyst

Spark SQL provides a SQL interface and DataFrame API. Catalyst is Spark SQL's query optimization framework. It analyzes logical plans and applies transformations before producing a physical execution plan.

Common optimizations include predicate pushdown, column pruning, constant folding, join planning, and expression simplification.

6. Shuffles

A shuffle redistributes data between executors. Operations such as groupBy, distinct, repartition, and many joins can require a shuffle.

Shuffles are expensive because they involve network transfer, serialization, disk I/O, and coordination. Reducing unnecessary shuffles is a major Spark performance principle.

7. Fault Tolerance

Spark can recompute lost partitions from lineage for RDD-based processing. In DataFrame workloads, Spark's execution model can also retry failed tasks and reconstruct intermediate data when possible.

Task failures do not necessarily mean the whole application has failed. Spark retries failed tasks according to configuration before ultimately failing a stage or job.

8. Spark Deployment Concepts

Common cluster managers include Spark Standalone, YARN, Kubernetes, and cloud-managed Spark platforms. The driver may run inside or outside the cluster depending on deployment mode.

Executors have configurable CPU and memory resources. Poor sizing can lead to slow execution, out-of-memory errors, excessive garbage collection, or underutilized resources.

9. Example PySpark Pipeline

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col

spark = SparkSession.builder.appName("DailySales").getOrCreate()

sales = spark.read.parquet("s3://bucket/raw/sales/")
result = (
    sales
    .filter(col("amount") > 0)
    .groupBy("customer_id")
    .sum("amount")
)

result.write.mode("overwrite").parquet("s3://bucket/curated/customer_sales/")
```

10. Key Terms for RAG Testing

Spark, driver, executor, task, stage, job, partition, shuffle, lineage, lazy evaluation, DataFrame, Catalyst, AQE, broadcast join, repartition, coalesce, caching, persistence, serialization, executor memory.